

Personal Productivity

Dashboard — Full Stack Task

Role	Full Stack Developer (3+ yrs exp)	Stack	React · Node.js · Express · MongoDB · Tailwind
Duration	1–2 Days	Submission	GitHub Repo + Live Demo (optional)
Type	Take-Home Assignment	Level	Senior / Mid-Senior

1. Project Overview

You are required to build **FocusFlow** — a Personal Productivity Dashboard that allows users to manage their tasks with a Kanban-style board. The app must be a full-stack application with a React frontend, a Node.js + Express REST API backend, and a MongoDB database. Authentication, task management, real-time drag-and-drop, and team collaboration features must be implemented.

2. Required Features

2.1 User Authentication

- JWT-based login & signup with refresh token rotation
- Password hashing using **bcrypt** (salt rounds ≥ 12)
- Protected routes using middleware on both client and server
- Persist auth state using **httpOnly cookies** (not localStorage)

2.2 Task Management

- Create, Read, Update, Delete (CRUD) tasks
- Task fields: title, description, priority (High/Medium/Low), status, due date, tags/categories
- Soft delete with **isDeleted** flag — no hard DB deletes
- Pagination on task list API (limit, page, sort params)

2.3 Kanban Board with Drag & Drop

- Three columns: **To Do** → **In Progress** → **Done**
- Drag & drop to move cards between columns (use **@dnd-kit/core**)
- Optimistic UI update on drag — revert on server error
- Column task counts update in real time

2.4 Categories & Tags

- Users can create custom categories with a color label
- Tasks support multiple tags; filter board by tag
- Tag statistics visible on dashboard (count per tag)

2.5 Due Dates & Reminders

- Date picker for due dates on each task
- Visual indicator for overdue tasks (red highlight)
- Due-soon badge for tasks within 48 hours
- Sort tasks by due date on the list view

2.6 Comments & Notes

- Threaded comments per task with timestamp
- Markdown support in task description and comments
- Edit and delete own comments only
- Comment count shown on task card

3. Tech Stack Requirements

Layer	Technology	Notes
Frontend	React 18 + Vite	Hooks only — no class components
Styling	Tailwind CSS v3	Custom design tokens in tailwind.config.js
State Mgmt	Zustand + React Query	Server state via React Query, UI state via Zustand
Routing	React Router v6	Nested routes, protected routes via outlet
Backend	Node.js + Express	MVC pattern, async/await throughout
Database	MongoDB + Mongoose	Indexed fields, lean queries for list endpoints
Auth	JWT + bcrypt	Access token (15m) + refresh token (7d)
Validation	Zod (FE) + Joi (BE)	Schema validation on both sides
Testing	Vitest + Supertest	Min. 5 unit tests + 3 API integration tests

4. API Endpoints (Expected)

Auth

POST	/api/auth/register	Register new user
POST	/api/auth/login	Login, returns access + refresh token
POST	/api/auth/refresh	Rotate refresh token
POST	/api/auth/logout	Invalidate refresh token

Tasks

GET	/api/tasks	List tasks (paginated, filterable)
POST	/api/tasks	Create new task
GET	/api/tasks/:id	Get single task with comments
PATCH	/api/tasks/:id	Update task fields
PATCH	/api/tasks/:id/status	Move task to new column
DELETE	/api/tasks/:id	Soft delete task

Comments

GET	/api/tasks/:id/comments	List comments for task
POST	/api/tasks/:id/comments	Add comment
DELETE	/api/comments/:commentId	Delete own comment

Tags

GET	/api/tags	List user tags with task counts
POST	/api/tags	Create tag with color
DELETE	/api/tags/:id	Delete tag (unassigns from tasks)

5. Expected Folder Structure

```
focusflow/
├── client/ # React + Tailwind
│   ├── src/
│   │   ├── components/ # Reusable UI components
│   │   ├── pages/ # Route-level components
│   │   ├── hooks/ # Custom hooks (useTasks, useAuth)
│   │   ├── store/ # Zustand stores
│   │   └── lib/ # api.js, utils.js
│   ├── tailwind.config.js
│   └── server/ # Node + Express
│       ├── src/
│       │   ├── controllers/ # Route handlers
│       │   ├── models/ # Mongoose schemas
│       │   ├── routes/ # Express routers
│       │   └── middleware/ # auth, error, validate
│       ├── utils/ # helpers, jwt utils
│       ├── .env.example
│       └── docker-compose.yml # Optional
└── README.md # Setup instructions (required)
```

6. Evaluation Criteria

Code Quality & Architecture	40%	Clean, modular MVC/component structure. No God components. DRY principles. Proper error handling with consistent HTTP status codes.
Feature Completeness	25%	All 6 required features implemented and working. Edge cases handled (empty states, loading, errors).
UI/UX Polish	15%	Tailwind usage shows design awareness — spacing, typography, responsive layout, hover/focus states.
API Design	10%	RESTful conventions followed. Request validation. Meaningful error messages. Pagination implemented correctly.
Testing	10%	At least 5 unit tests and 3 API integration tests. Tests cover happy path and error cases.

7. Bonus Points (Optional)

★ Real-time updates	WebSockets (Socket.io) for live task status sync across tabs
★ Dark mode	Tailwind dark: variant with persistent preference via localStorage
★ Analytics page	Task completion rate chart using Recharts
★ Docker setup	docker-compose.yml for one-command local setup
★ Deployment	Live URL (Vercel + Railway / Render)

8. Submission Instructions

1	Push your code to a public GitHub repository named focusflow-[yourname]
2	Include a detailed README.md : setup steps, env variables, how to run tests
3	Add a .env.example file (never commit actual secrets)
4	Record a short 2–3 minute Loom walkthrough of the working app
5	Submit the GitHub URL (+ Loom link) to your hiring contact



Important Notes: Do not use any boilerplate generators or full-stack starter kits (e.g. create-t3-app, MERN boilerplate). We want to see your architecture decisions. Using AI tools (GitHub Copilot, ChatGPT) is permitted but you must be able to explain every line of code during the follow-up interview. Plagiarism of another candidate's submission will result in immediate disqualification.